BRAC UNIVERSITY

Inspiring Excellence

# CSE471 : System Analysis and Design
# Lab Assignment on API Postman Collection

**Project Title : ShebaConnect - An Integrated Government Service Navigation & Assistance Platform**

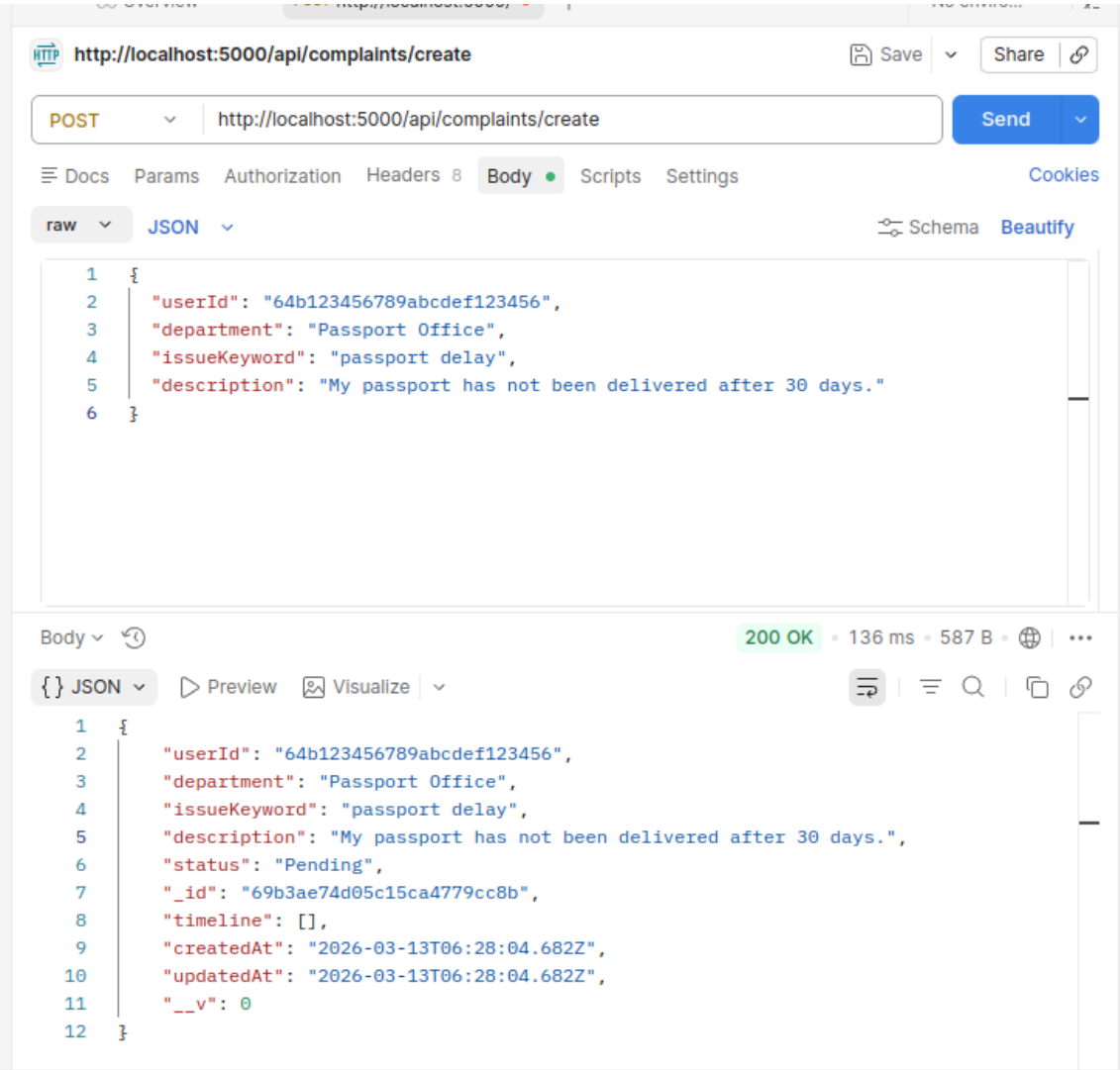
**Group Number :** 06
**Backend Framework :** Node.js + Express.js

**Submitted by** : Sujit Kumar Datta
**Student ID :** 22201946
**Section:** 05

**Date : 14/03/2026**

# APIs of Feature - 01

```
router.post('/create', async (req, res) => {
  try {
    console.log("Received data:", req.body);

    const requiredFields = [
      'userId',
      'citizenName',
      'citizenId',
      'contactNumber',
      'department',
      'issueKeyword',
      'description'
    ];

    for (const field of requiredFields) {
      if (!req.body[field]) {
        return res.status(400).json({
          message: `Missing required field: ${field}`
        });
      }
    }

    const timeline = [
      {
        status: "Pending",
        comment: "Complaint submitted successfully",
        updatedBy: req.body.citizenName || "Citizen",
        date: new Date()
      }
    ];

    const date = new Date();
    const year = date.getFullYear().toString().slice(-2);
    const month = (date.getMonth() + 1).toString().padStart(2, '0');
    const random = Math.floor(Math.random() * 10000).toString().padStart(4, '0');
    const complaintNumber = `CMP${year}${month}${random}`;

    const complaintData = {
      userId: req.body.userId,
      citizenName: req.body.citizenName,
      citizenId: req.body.citizenId,
      contactNumber: req.body.contactNumber,
      email: req.body.email || "",
      address: req.body.address || "",
      department: req.body.department,
      issueKeyword: req.body.issueKeyword,
      description: req.body.description,
      priority: req.body.priority || "medium",
      status: "Pending",
      complaintNumber: complaintNumber,
      timeline: timeline
    };
```



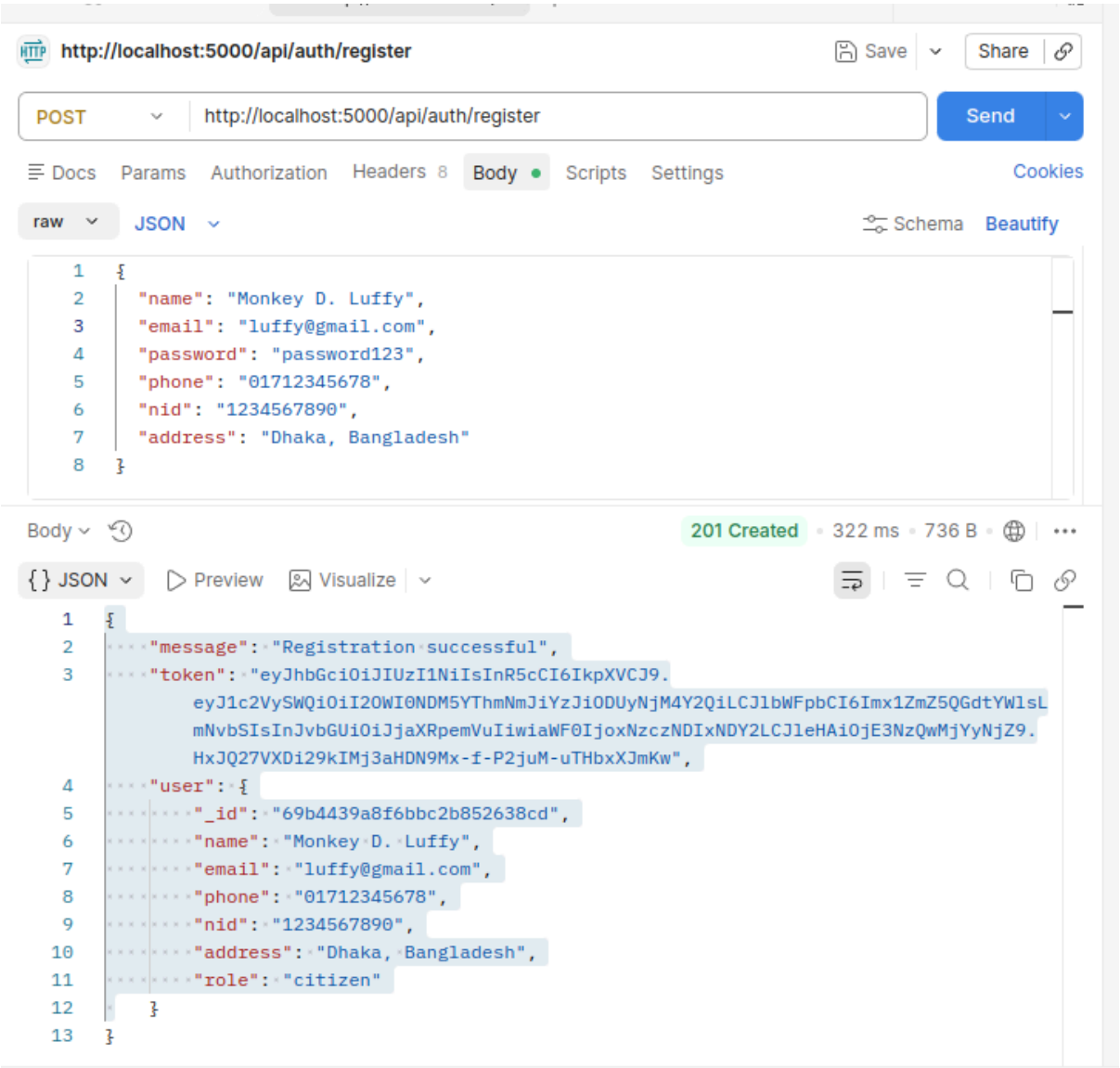

- Endpoint URL: http://localhost:5000/api/complaints/create
- HTTP Method: POST
- Headers: Content-Type: application/json
- Body: {

```
    "userId": "64b123456789abcdef123456",
    "department": "Passport Office",
    "issueKeyword": "passport delay",
    "description": "My passport has not been
delivered after 30 days.",
    "status": "Pending",
    "_id": "69b3ae74d05c15ca4779cc8b",
    "timeline": [],
    "createdAt": "2026-03-13T06:28:04.682Z",
    "updatedAt": "2026-03-13T06:28:04.682Z",
    "__v": 0
}
```

# APIs of Feature - 02

```
// REGISTER
router.post('/register', async (req, res) => {
  try {
    const { name, email, password, phone, nid, address } = req.body;

    const existingUser = await User.findOne({
      $or: [{ email }, { phone }, { nid }]
    });

    if (existingUser) {
      if (existingUser.email === email) {
        return res.status(400).json({ message: 'Email already registered' });
      }
      if (existingUser.phone === phone) {
        return res.status(400).json({ message: 'Phone number already registered' });
      }
      if (existingUser.nid === nid) {
        return res.status(400).json({ message: 'NID already registered' });
      }
    }

    const salt = await bcrypt.genSalt(10);
    const hashedPassword = await bcrypt.hash(password, salt);

    const user = new User({
      name,
      email,
      password: hashedPassword,
      phone,
      nid,
      address,
      role: 'citizen'
    });

    await user.save();

    const token = jwt.sign(
      { userId: user._id, email: user.email, role: user.role },
      process.env.JWT_SECRET || 'your-secret-key',
      { expiresIn: '7d' }
    );

    const userData = {
      _id: user._id,
      name: user.name,
      email: user.email,
      phone: user.phone,
      nid: user.nid,
      address: user.address,
      role: user.role
    };

    res.status(201).json({
      message: 'Registration successful',
      token,
      user: userData
    });
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS



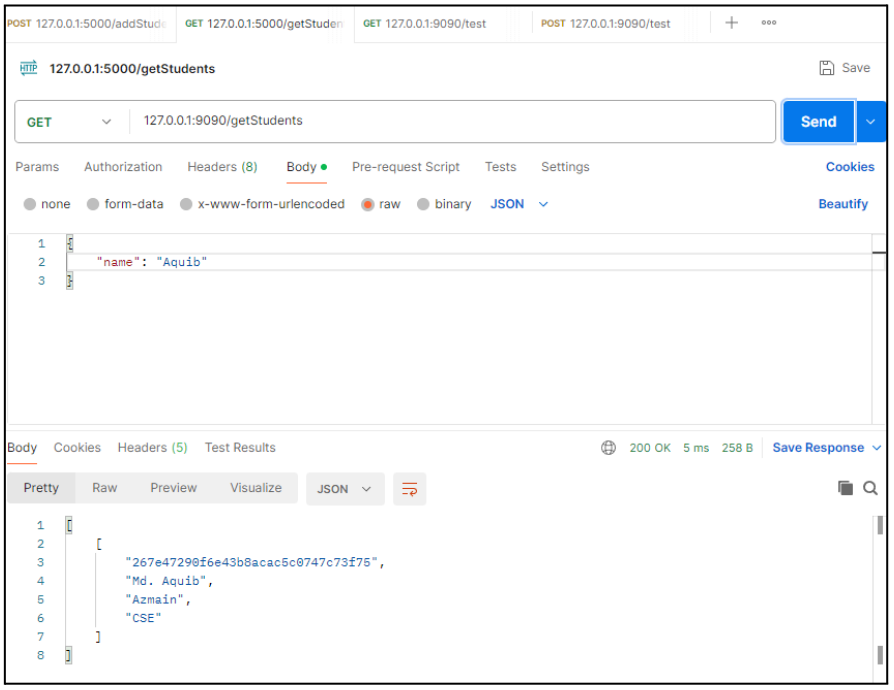

- Endpoint URL: http://localhost:5000/api/auth/register
- HTTP Method: POST
- Headers: Content-Type: application/json
- Body: {
"message": "Registration successful",
"token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySWQiOiI2OWI0NDM5YThmNmJiYzJiODUyNjM4Y2QiLCJlbWFpbCI6Imx1ZmZ5QGdtYWlsLmNvbSIsInJvbGUiOiJjaXRpemVuIiwiaWF0IjoxNzczNDIxNDY2LCJleHAiOjE3NzQwMjYyNjZ9.HxJQ27VXDi29kIMj3aHDN9Mx-f-P2juM-uTHbxXJmKw",
"user": {
"_id": "69b4439a8f6bbc2b852638cd",
"name": "Monkey D. Luffy",

```
        "email": "luffy@gmail.com",
        "phone": "01712345678",
        "nid": "1234567890",
        "address": "Dhaka, Bangladesh",
        "role": "citizen"
    }
}
```

Instruction:

1. You have to develop all the REST APIs/Endpoints of two features (which interfaces you have submitted in the Figma assignment) of your project using your preferred backend framework. Database must be connected.
2. Each API description will include:
   - Endpoint URL (e.g., https://api.example.com/users)
   - HTTP Method (GET, POST, PUT, DELETE, etc.)
   - Headers (like Content-Type or Authorization)
   - Body/Parameters (data to send with request)
3. This is an individual submission. The developed APIs can not be duplicated among group members.
4. Add a code snippet for each API.
5. Run a backend server on a specific port. The port number should be the **last four digits of your student id**.
6. Test your developed API in Postman. Add a screenshot for each API.
7. For example:

```python
@app.route('/getStudents', methods=['GET'])
def getStudents():
    c = sqlite3.connect("student.db").cursor()
    c.execute("SELECT * FROM STUDENTS")
    data = c.fetchall()
    return data
```



- Endpoint URL: 127.0.0.1:9090/getStudetns
- HTTP Method: Get
- Headers: None
- Body: {“name” : “Aquib”}